



Project 2

Note:

To write assembly program you can use any text editor such as Notepad. At this point, the structure of the assembly program should be similar to the structure below. You can add your code as mentioned below inside the structure. Also, in assembly program, any line that starts with `#` will be considered as a comment and the assembler will not even execute it.

```
# you need to write the two lines below at the beginning of your program

.text
.globl main

main:

#===== You can write your assembly code here

# =====

# You need to write the following commands at the end of your program to
# skip running the program

li    $v0,10
syscall
```

Figure 1: Simple structur of the assembly program.

Question 1

[12 points] A compact digital speed and acceleration sensor is used to measure both the speed and acceleration of a moving system simultaneously. The sensor outputs a 32-bit binary signal, where the first 20 bits represent the speed reading, and the remaining bits represent the acceleration reading. Three readings from this sensor, given in hexadecimal format, are provided below: 12344532 , 23c1ecd2 , ffa1234

Write MIPS assembly code segments to perform the following operations:

1. [2.5 points] Store the three sensor readings in memory, starting from memory address 0x10010000.
2. [2.5 points] Separate the speed readings from the acceleration readings. Each reading should be 32 bits in size.
3. [2.5 points] Store the three speed readings in memory, starting from address 0x10010020.
4. [2.5 points] Store the three acceleration readings in memory, starting from address 0x10010040.

Both speed and acceleration readings should remain in the same order as the original combined readings obtained from the sensors.

Example: The first compact reading from the sensor is 12344532h, which is stored in memory starting at address 0x10010000. Remember, four memory locations are required to store this data. The first 20 bits, 44532h, should be stored in memory starting from address 0x10010020. Again, four memory locations are required to store this data. The last 12 bits, 123h, should be stored in memory starting from address 0x10010040. As before, four memory locations are required to store this data.

Simulate your code using QtSpim to assure that it works properly. Name your code Prj2.Q1.s and submit .s file via UBlearn.

Question 2

[10 Points] Write MIPS assembly code that is used to convert centimeter (cm) to meter (m). As you know the 1 m = 100 cm, therefore, to convert from centimeter to meter we need to divide the centimeter value by 100 as shown below:

$$m = \frac{cm}{100} \quad (1)$$

Write MIPS assembly code to convert the length which is given in cm to meter. Note that the length can be fractional number, and it is not always an integer.

You will need to perform the following steps:

1. Convert decimal 100.0 to its equivalent single precision binary representation and store the result into any INT register, for example, t0. Note that the value inside t0 now is not (100.0) because t0 is not used with fractional numbers, but we will use it to store the binary of 100.0 as an integer number.
2. Store t0 in the memory location starts at 0x10010000
3. Load 100.0 from the memory (which is stored in the previous step) into any FP register, say f0 using "lwc1" instruction.

4. Let us assume we need to convert 350.0 cm to meter. Now we need to store the 350.0 cm into any FP register. To do that follow the following steps:
 - * Convert 350.0 to its single precision binary equivalent and store it in any INT register as a binary integer number, say t1
 - * Now store t1 (i.e 350.0) into the memory, for example use memory location starts at 0x10010010
 - * Use "lwc1" instruction to load 350.0 from memory into any FP register, say f1
 - * Now use "div.s" instruction to perform the division (f1/f0) and store the result in f2 register.
- Simulate your code using QtSpim to assure that it works properly. Name your program Prj1_Q2.s and submit .s file via UBlearn.